HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

IPC & scheduling cheat sheet

Practical 03 | Everything runs in the browser Linux; chapter 5 and the last step of chapter 6 are worked out on paper.

01 Pipes

`A | B`

Every command is its own process; the pipe carries the output of A to the input of B.

```
grep ALARM logs/vitals.log | cut -d' ' -f2 | sort -u
```

cut -d' ' -f2: second column · sort -u: sort and drop duplicates.

`mkfifo`

A named pipe: a pipe with a name, usable by any two processes. `ls -l` shows a `p`.

```
mkfifo ~/ipc/alarms
echo 'bed-07 ALARM' > ~/ipc/alarms &   writer in the background
cat ~/ipc/alarms                       reader
```

A writer waits until a reader opens the pipe. Without & the shell waits forever.

02 Sockets

Two-way connection: the client sends a request, the server answers. Local: a Unix domain socket file; across machines: a network address and port.

```
echo "bed-12 spo2=86 ALARM" | socat - UNIX-CONNECT:ipc/station.sock
netstat -lx                            who is listening (Unix sockets)
```

Mechanism	Direction	Reach	Typical use
pipe |	one way	one machine	chain commands
named pipe	one way	one machine	two unrelated programs
message queue	many → many	one machine*	jobs, events
shared memory	all share	one machine	large data, fastest
socket	two way	local or network	servers, web, APIs

* brokers such as MQTT extend queues across the network.

03 Shared data, locks, semaphores

Read, change, write: two processes that both read the same old value lose an update (race condition).

```
admit ward-a 10 & admit ward-b 10 &   at the same time
wait                                  wait for all background jobs
cat ~/ward/admitted                   less than 20!
```

Lock (mutual exclusion): only one at a time in the critical section. `flock FILE` is the Linux tool. A lock only helps if **every** process uses it.

Semaphore S = number of free units

P(S): S > 0 → S − 1 and continue, otherwise wait.
V(S): someone waits → hand the unit over (S stays), otherwise S + 1.
S = 1 works like a lock, S = N lets up to N in. S never goes below 0.

04 Deadlock

Each job holds one lock and waits for the other: nobody ever continues. The jobs still exist, they just make no progress.

Condition	Meaning
mutual exclusion	a resource is used by one at a time
hold and wait	hold one resource, wait for another
no preemption	nobody takes a resource away
circular wait	a circle of waiting jobs

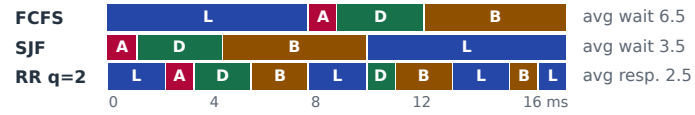
All four needed, break one. Easiest: every job takes the locks in the **same order**.

```
killall syncjob                        stop the stuck jobs politely
```

05 Scheduling by hand

Metric	Meaning
waiting time	ready but not running (no I/O: start time − arrival)
turnaround	finish − arrival
response time	first run − arrival

Ward jobs, all ready at 0: L 7, A 1, D 3, B 5 ms.



Algorithm	Idea	Strength	Weakness
FCFS	arrival order	simple, fair	convoy effect
SJF	shortest next	min. average wait	needs burst lengths, starvation
Round Robin	quantum q, then back	quick first turn	many context switches
Priority	most important first	urgent work first	starvation without aging

06 Scheduling in Linux

Fair shares by weight. nice −20 … 19: higher = nicer = smaller share. Users may only make processes nicer.

```
crunch a 3 & crunch b 3 & wait        two jobs share one core
nice -n 10 COMMAND                    start with nice 10
renice -n 19 -p PID                   change a running process
```

Share = weight ÷ sum of all weights. Weights: nice 0 = 1024, 5 = 335, 10 = 110, 19 = 15. Example: 335 ÷ (1024 + 335) ≈ 24.7 %.